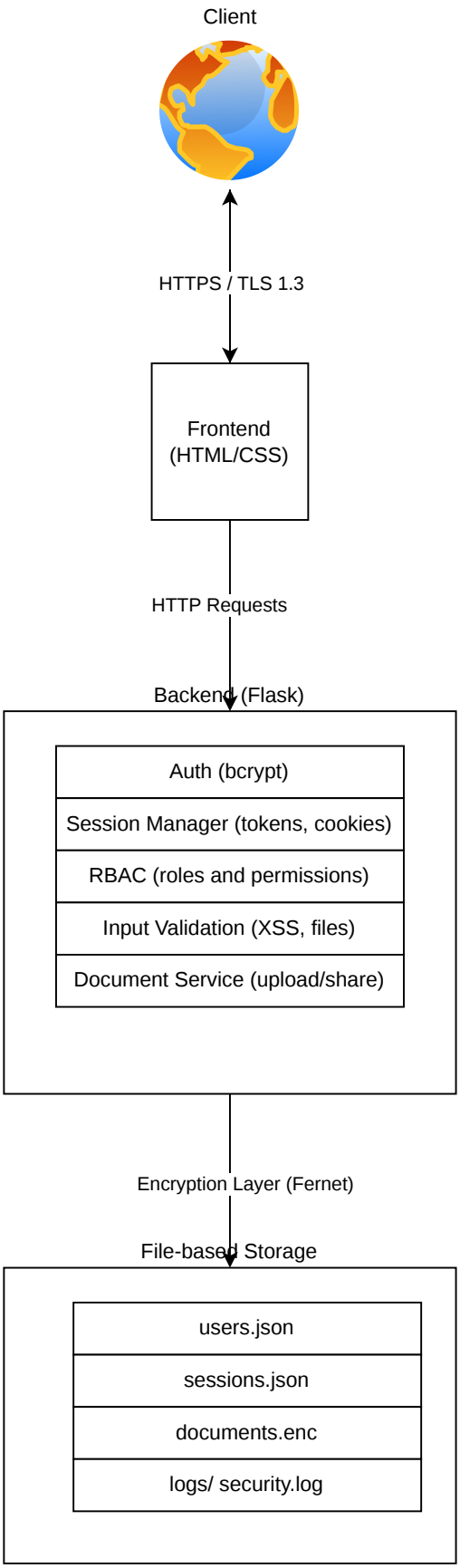
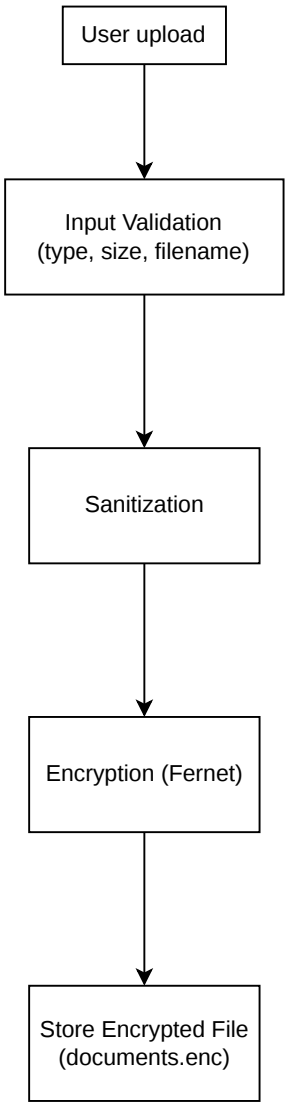
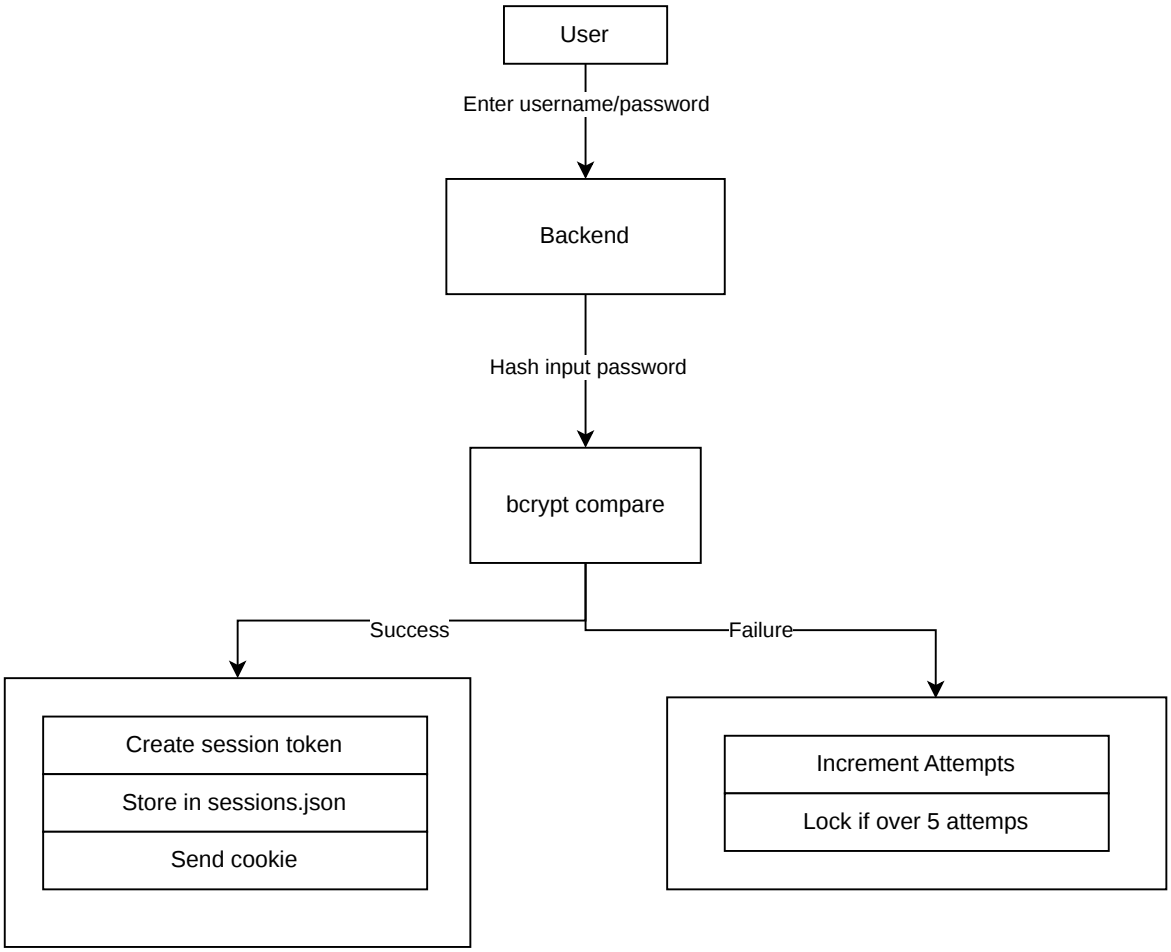
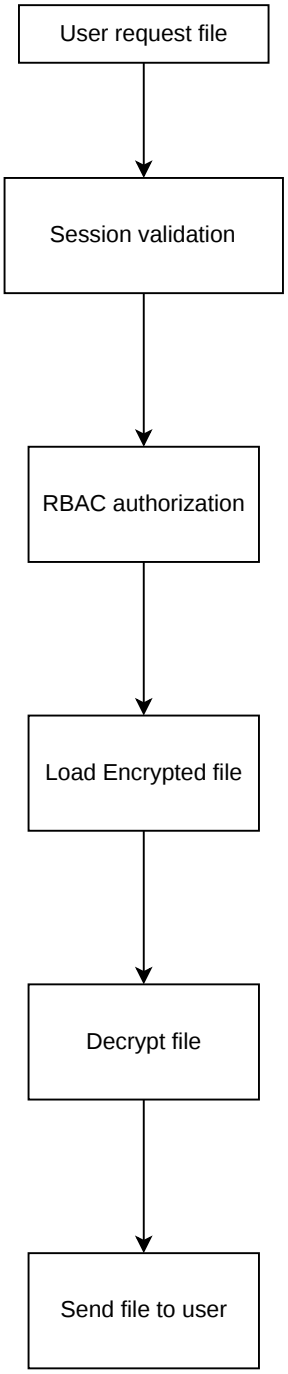
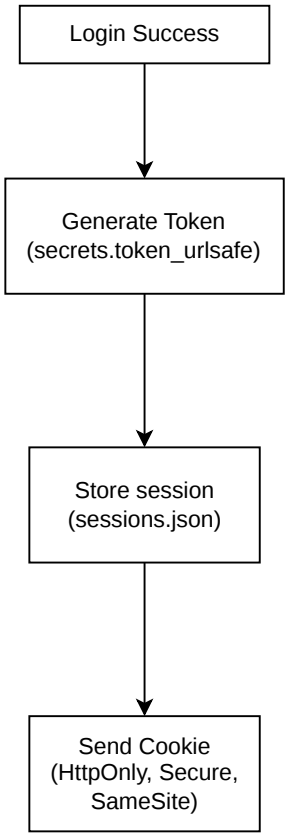

Client
HTTPS / TLS 1.3
Frontend
(HTML/CSS)
HTTP Requests
Backend (Flask)
Auth (bcrypt)
Session Manager (tokens, cookies)
RBAC (roles and permissions)
Input Validation (XSS, files)
Document Service (upload/share)
Encryption Layer (Fernet)
File-based Storage
users.json
sessions.json
documents.enc
logs/ security.log

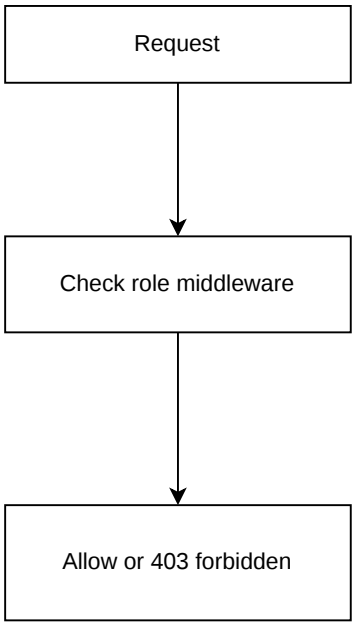

User upload
Input Validation
(type, size, filename)
Sanitization
Encryption (Fernet)
Store Encrypted File
(documents.enc)

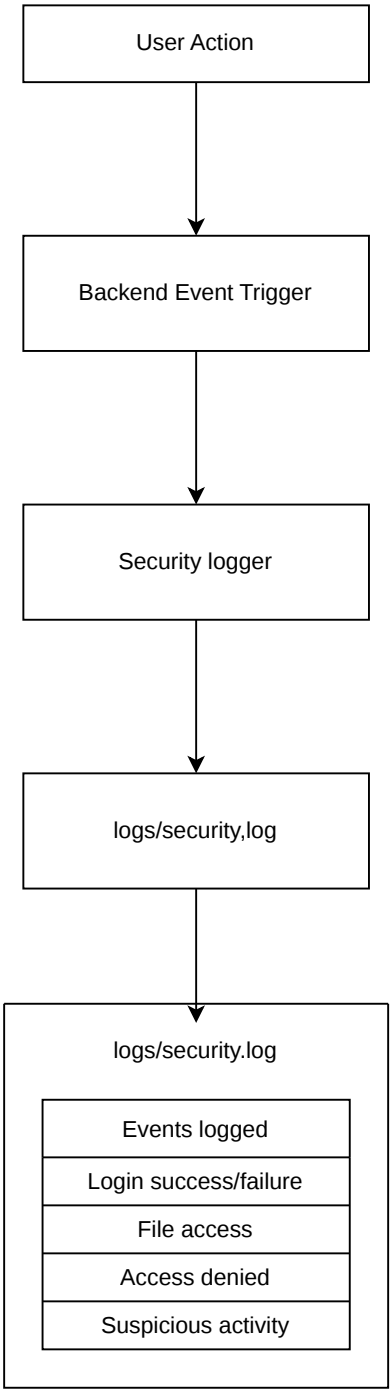

User
Enter username/password
Backend
Hash input password
bcrypt compare
Success
Failure
Create session token
Store in sessions.json
Send cookie
Increment Attempts
Lock if over 5 attemps


User request file
Session validation
RBAC authorization
Load Encrypted file
Decrypt file
Send file to user


Login Success
Generate Token
(secrets.token_urlsafe)
Store session
(sessions.json)
Send Cookie
(HttpOnly, Secure,
SameSite)

	User Roles		
	Admin	User	Guest
Create content	✓	✓	x
Edit own content	✓	✓	x
Delete content	✓	✓	x
View all content	✓	x	x
Manage users	✓	x	x
View shared	✓	✓	✓




User Action
Backend Event Trigger
Security logger
logs/security,log
logs/security.log
Events logged
Login success/failure
File access
Access denied
Suspicious activity

A.4 Technology stack justification

- Flask: Chosen for its simplicity and built-in handling of routing, HTTP requests, and secure cookie-based sessions.
- Werkzeug (secure_filename):. It sanitizes uploaded filenames to prevent Directory Traversal attacks.
- Bcrypt: Specifically designed to be computationally slow and includes automatic salting, making it highly resistant to brute-force and rainbow table attacks.
- Cryptography (Fernet): Provides symmetric encryption to uploaded files, encrypting them at rest

B.1 Asset Identification

Asset Name
User creentials
Document Data (at rest)
Session tokens
Cryptographic keys
Security audit logs

B.2 Threat Enumeration

Category	Specific Threat	Mitigation
Spoofing	Attacker steals session cookie	HttpOnly, Secure, SameSite='Strict' flags
Tampering	User modifies and encrypted file on disk	Fernet's built-in HMAC (authentication encryption)
Repudiation	User denies uploading a malicious file	security.log records UserID and Filename
Info discolsure	Attack steals users.json	Bcrypt makes hashes computationally expensive to crack
Denial of Service	Attack crashes log view	Log tailing (f.readlines() [-50:]) saves memory
Elevation of Privlege	Guest attempts to access /upload	@require_role decorate enforces least privilege

B.3 Vulnerability Assessment

- Race Conditions: Because the system uses users.json instead of a relational database with ACID properties, concurrent registration or login attempts can potentially cause race conditions, leading to data corruption in the JSON file.
- Self-Signed Certificates: Because the app is using self-signed certificates (cert.pem, key.pem) there is no trusted Certificate Authority (CA). If a user is conditioned to bypass browser security warnings, an active attacker on the same network could perform a man-in-the-middle attack.
- In-Memory Key Exposure: The ENCRYPTION_KEY is stored as an environment variable and loaded into the Flask application's RAM. If an attacker can initiate a memory dump, they could potentially extract the Fernet symmetric key and decrypt the .enc files stored on disk.

B.4 Attack Scenarios

- Automated Credential Stuffing
  - Attack: An attacker acquires a breached password list and uses an automated script to continuously submit POST requests to the /login endpoint.
  - System Response: The Auth Manager tracks failed attempts and after the number of failed attempts reaches 5, the account gets locked out for 15 minutes. Scenario B: Directory Traversal through Malicious Upload
  - Attack: An authenticated user attempts to upload a file named ../../../data/users.json to overwrite the application's credential database
  - System Response: The application passes the input through secure_filename(), stripping the ../ sequences. The file is safely renamed to users.json and encrypted.
- Scenario C: Privilege Escalation
  - Attack: A user assigned the 'Guest' role manually modifies the URL to navigate to /admin_dashboard to view sensitive audit logs.
  - System Response: The @require_role('Admin') decorator intercepts the request, checks the cryptographically signed session cookie, realizes the roles do not match, and returns HTTP 403 Forbidden error.

B.5 Risk Prioritization

Risk	Threat Scenario	Likelihood	Impact
1	Path Traversal	High	Critical
2	Session Hijacking / XSS	High	High
3	Brute-force	High	Medium
4	Data Exposure through Disk Theft	Low	Critical
5	JSON Concurrency Corruption	Low	Medium

C.1 Authentication & Account Lockouts

- **Control description:** The system enforces a defense-in-depth approach to authentication, preventing credential stuffing and unauthorized access.
- **Implementation approach:** Passwords are hashed using Bcrypt (12 rounds) prior to storage in users.json. A custom tracking mechanism records failed logins. Upon 5 failed attempts, the user account is programmatically locked for 15 minutes.
- **Testing methodology:** Attempted to log in 6 consecutive times with incorrect credentials for a known user. Verified that the 6th attempt, even with the correct password, returned an "Account locked" error.
- **Known limitations:** A distributed brute-force attack across many different usernames could overwhelm the system before individual lockouts trigger.
- **Mitigation strategies:** integrating a CAPTCHA.

C.2 Role-Based Access Control (RBAC)

- **Control description:** Applying the principle of least privilege by ensuring users can only access endpoints authorized for their specific role.
- **Implementation approach:** Used Python decorator (@require_role) wraps to protect Flask routes. Doing so intercepts the request, checks the user's role that's stored in the signed session cookie, and allows or denies access based on a whitelist.
- **Testing methodology:** Logged in as a 'Guest' user and attempted to go directly to the /admin_dashboard URL and verified the system returned a 403 Forbidden error.
- **Known limitations:** A session cookie could be stolen through XSS, and the attacker can use the victim's role, bypassing the RBAC check.
- **Mitigation strategies:** Strict enforcement of HttpOnly and SameSite cookie flags

C.3 Input Validation

- **Control description:** Prevents attackers from using malformed filenames to escape the designated upload directory and interact with sensitive server files.
- **Implementation approach:** All user-uploaded files are processed through the Werkzeug library's secure_filename() function.
- **Testing methodology:** Uploaded a file named ../../../data/users.json via a modified HTTP POST request and verified that the file was safely saved in the uploads folder as users.json instead of overwriting.
- **Known limitations:** secure_filename() only sanitizes the file's name and not its contents, so a user could still upload a malware infected and encrypted file.
- **Mitigation strategies:** Integrating an antivirus scanning API to analyze the plaintext buffer before encryption and storage.

D.1 Encryption at Rest

- **Control description:** Protects the confidentiality of documents stored on server disk.
- **Implementation approach:** Uploaded files are encrypted in RAM through the cryptography.fernet library. The resulting ciphertext is saved as a .enc file.
- **Testing methodology:** Manually opened a .enc file in a text editor directly from the server's hard drive and verified that the contents were some unreadable ciphertext.
- **Known limitations:** If the server's RAM is compromised during a file transfer, the plaintext data and the encryption key could be exposed.
- **Mitigation strategies:** Deploying the application within an isolated VM.

D.2 Encryption in Transit

- **Control description:** Prevents network eavesdropping and Man-in-the-Middle attacks.
- **Implementation approach:** The Flask application is configured to run on HTTPS by providing an ssl_context with self-signed certificates (cert.pem, key.pem).
- **Testing methodology:** Used Wireshark to capture packets between the browser and the server and verified that payload data + session cookies were fully encrypted.
- **Known limitations:** Self-signed certificates trigger browser warnings and do not guarantee the server's identity to the user which can then lead to alert fatigue.
- **Mitigation strategies:** Obtaining a certificate from a trusted Certificate Authority (CA)

D.3 Key Management

- **Control description:** Ensure that cryptographic keys are not exposed in source code.
- **Implementation approach:** Relevant keys are stored in a .env file and included in .gitignore
- **Testing methodology:** Double-checking repository to verify that no .env files or hardcoded key strings.
- **Known limitations:** The .env file is still stored in plaintext on the production server's disk, accessible to anyone with local admin’s rights.
- **Mitigation strategies:** Utilizing a dedicated secrets management service such as AWS Secret Manager